IST604 – Web Services & Distributed Computing

Chapter 3: Developing Web Services Using SOAP — Lecture Summary

3.1 Recap of Core Standards

3.1.1 SOAP Message Transmission

SOAP message delivery can pass through **intermediaries** before reaching the final destination. Three roles are defined:

Role	Description
SOAP Sender	Creates and sends the SOAP message
SOAP Intermediary	Optional node(s) that intercept and process messages (filtering, logging, caching, etc.)
Ultimate SOAP Receiver	The intended final destination of the SOAP message

The path is: `SOAP Sender → [Intermediary → Intermediary → ...] → Ultimate SOAP Receiver`

3.1.2 WSDL (Web Services Description Language)

- An XML document describing a web service's method names, parameters, and access details
- Defines the binding mechanism linking protocols, data formats, and service endpoints
- Serves as the **metadata language** in the web services model — describing what the service offers, where it is, and how to access it

3.1.3 UDDI (Universal Description, Discovery, and Integration)

- An XML-based framework for describing, discovering, and integrating web services
- A directory of web service interfaces described by WSDL
- Used by service brokers to register providers; service requesters query UDDI to locate and invoke services

---

3.2 SOAP-Based Web Services Implementation

3.2.1 Typical Scenario — Request/Response Pattern

- The **client's SOAP library** packages and sends a SOAP request message
- The **service's SOAP library** processes it and sends back a SOAP response message
- Both sides are abstracted behind their respective SOAP libraries — developers work with Java objects, not raw XML

3.2.2 Implementation Structure (Best Practice)

A SOAP-based web service can technically be a single Java class, but best practice separates it into two components:

Component	Acronym	Role
Service Endpoint Interface	SEI	Java interface declaring the web service operations (methods)
Service Implementation Bean	SIB	Java class implementing the SEI methods

The SIB can be implemented as either:

- **POJO** (Plain Old Java Object) — a regular Java class instance
- **Stateless Session EJB** (Enterprise Java Bean) — for use in a full Java Application Server, providing managed, scalable, and secure services

---

3.3 JAX-WS Implementation with Java SE

For simple SOAP services, **Java SE includes a built-in HTTP server** — no external application server is required for development and light production use.

Key Annotations

Annotation	Applied To	Purpose
`@WebService`	Interface (SEI) and Implementation class (SIB)	Marks the class/interface as a web service
`@SOAPBinding(style = SOAPBinding.Style.RPC)`	Interface	Specifies RPC-style SOAP binding
`@SOAPBinding(style = SOAPBinding.Style.DOCUMENT)`	Interface	Specifies Document-style SOAP binding (industry default)
`@WebMethod`	Methods in SEI	Exposes the method as a web service operation

Publishing a Web Service

Use `Endpoint.publish()` in the main method:

```
Endpoint.publish("http://127.0.0.1:PORT/path", new MyImplementation());
```

- First argument: the publication URL
- Second argument: an instance of the SIB

Testing a Deployed Service

Access the auto-generated WSDL contract in a browser:

```
http://localhost:PORT/path?wsdl
```

Common Runtime Issue

On JDK 11+, the `@WebServiceProvider` annotation API is not bundled by default. Solution: download the **javax.annotation-api** JAR from Maven Repository and add it to the project classpath.

3.4 Worked Examples (RPC Style)

Structure of All Examples

Every JAX-WS RPC example follows the same 4-file pattern:

File	Role
`XxxInterface.java`	SEI — declares `@WebMethod` operations
`XxxImplementation.java`	SIB — implements the operations
`XxxWS.java`	Publisher — calls `Endpoint.publish()`
`XxxClient.java`	Consumer — calls the service via a proxy

Example 1: TimeServer (RPC Style)

- Methods: `getTimeAsString()` → returns current time as a String; `getTimeAsElapsed()` → returns Unix epoch milliseconds
- Demonstrates: converting a local Java program into a web service
- Clients written in both **Perl** (using `SOAP::Lite`) and **Java** (using `Service.create()` + `QName`)

Example 2: HelloWorld (RPC Style)

- Method: `getHelloWorldAsString(String name)` → returns `"Hello " + name`
- Demonstrates: passing a parameter to a web service operation

Example 3: Calculator (RPC Style)

- Method: `add(int a, int b)` → returns the sum
- Demonstrates: numeric parameters and return types in a web service

Java Client Pattern (All RPC Examples)

```
URL url = new URL("http://localhost:PORT/path?wsdl");
QName qname = new QName("namespace", "ServiceName");
Service service = Service.create(url, qname);
XxxInterface port = service.getPort(XxxInterface.class);
```

```
// Call methods on port as if local
port.someMethod(args);
```

3.5 The Role of the WSDL File

In JAX-WS, the WSDL is the **formal contract** between the service provider and the consumer.

WSDL Role	Description
Service Roadmap	Lists all available operations (e.g., `add`, `placeOrder`)
Data Structure Definition	Defines expected/returned types using XML Schema (XSD)
Connectivity Details	Provides the endpoint URL and protocol (SOAP over HTTP)
Automation	Machine-readable; tools like `wsimport` generate Java stubs automatically

---

3.6 wsgen and wsimport

Two JDK command-line tools for generating supporting artifacts (code) for SOAP web services:

Feature	`wsgen`	`wsimport`
Approach	**Bottom-Up** (Code-First)	**Top-Down** (Contract-First)
Primary Input	Compiled `.class` file annotated with `@WebService`	A `.wsdl` file (local or URL)
Typical User	**Service Provider** (developer building the service)	**Service Consumer** (client developer)
Key Output	JAXB classes, optional WSDL file	Client stubs: SEI, Service class, JAXB beans

When to Use Each

- `wsgen`: When you need to share a physical WSDL file before deployment, or package it in an archive. Note: for simple testing, the JAX-WS runtime generates these artifacts dynamically when the endpoint is published.
- `wsimport`: When you are a client developer without access to the service source code — only the WSDL URL.

wsimport Command Syntax

```
wsimport -keep -s /path/to/src -p package.name http://host:port/service?wsdl
```

Flag	Meaning
`-keep`	Keeps generated Java source files (not just `.class` files)
`-s <dir>`	Output directory for generated source files
`-p <package>`	Target package name for generated classes

wsimport Client Pattern (3 Steps)

```
// 1. Initialize the generated Service class
CalculatorImplementationService service = new
CalculatorImplementationService();
// 2. Get the Port (proxy interface)
CalculatorInterface port = service.getCalculatorImplementationPort();
// 3. Call the method like a local Java method
int result = port.add(10, 25);
```

3.7 Document Style

RPC Style vs Document Style

Aspect	RPC Style (`Style.RPC`)	Document Style (`Style.DOCUMENT`)
Message structure	Mimics a function call — parameters mapped directly	Sends a full XML document as the message body
Data types	Limited to simple types (String, int, etc.)	Supports complex/rich data types and objects
Industry status	Simpler, but less flexible	**Industry standard** — default if omitted
Annotation	`@SOAPBinding(style = Style.RPC)`	`@SOAPBinding(style = Style.DOCUMENT)` or omit

Document style is the **default** in JAX-WS. If `@SOAPBinding` is omitted entirely, Document style is used.

Example: Order Processing Service (Document Style)

- Method: `placeOrder(String item, int quantity)` → returns order confirmation string
- Demonstrates: Document-style annotation, `Endpoint.publish()`, and two client approaches:
  - **Direct client** (`OrderClient1`): manually constructs `Service` using `QName` and WSDL URL
  - **wsimport client** (`OrderClient2`): uses auto-generated stubs for a cleaner, simpler client

---

3.8 Summary of Full Development Workflow

```
1. Define SEI (interface with @WebService, @WebMethod, @SOAPBinding)
        ↓
```

```
2. Implement SIB (class with @WebService(endpointInterface="..."))
        ↓
3. Publish with Endpoint.publish(URL, new SIBInstance())
        ↓
4. Test: browser → http://host:port/path?wsdl
        ↓
5. Client (Option A): Direct — Service.create(url, qname) + getPort()
   Client (Option B): wsimport → generate stubs → use generated Service class
```